

Prose or Structure?

Evaluating Security-Claim Generation in LLM-Based Code Analysis

Sebastian Grünewald, 6. Oktober 2026

1 Idee und Ziel

Sprachmodelle können Quellcode untersuchen und mögliche Sicherheitsprobleme beschreiben. Ihre Antworten bestehen häufig aus längeren Berichten. Wenn man diese Ergebnisse anschließend systematisch prüfen oder miteinander vergleichen möchte, braucht man jedoch einzelne, klar formulierte Aussagen. Solche Sicherheitsbehauptungen werden in dieser Arbeit als *Claims* bezeichnet.

Das Projekt untersucht zwei Wege, um aus demselben Quellcode zu solchen Claims zu gelangen. Beim ersten Weg schreibt das Modell zunächst einen Bericht. Aus diesem Bericht werden danach einzelne Aussagen herausgearbeitet. Beim zweiten Weg gibt das Modell seine Aussagen unmittelbar in einer vorgegebenen Struktur aus:

P — über einen Bericht: Code → Fließtextbericht → einzelne Claims

D — direkt strukturiert: Code → einzelne Claims im vorgegebenen Format

Beide Wege verwenden dasselbe Ausgabeformat. Bewertet wird, ob der bereitgestellte Code die Aussagen belegt, ob wichtige Sicherheitsaspekte erfasst werden und ob Voraussetzungen und Unsicherheit erhalten bleiben. Bei P kommt eine weitere Frage hinzu: Geben die extrahierten Claims den ursprünglichen Bericht richtig wieder?

Ein Claim kann den Bericht genau wiedergeben und trotzdem fachlich falsch sein. **Die Treue zum Bericht und die Richtigkeit im Code werden deshalb getrennt bewertet.** Das ZIP-Beispiel ab Seite 4 macht diesen Unterschied konkret.

Ziel ist eine überschaubare empirische Studie für ein erstes Paper. Sie soll zeigen, wie sich die beiden Wege auf die Qualität und den Aufwand der Claim-Erzeugung auswirken. Ein System, das sämtliche Sicherheitsbehauptungen automatisch auf ihre Wahrheit prüft, ist dafür zunächst nicht erforderlich.

2 Forschungsstand und möglicher Beitrag

Die Zerlegung längerer Texte in einzeln prüfbare Aussagen ist bereits Gegenstand bestehender Forschung. **DnDScore**, **DecompScore** und **VeriScore** untersuchen unter anderem, wie Aussagen zerlegt werden, welcher Kontext dabei erhalten bleiben muss und wie ihre Qualität bewertet werden kann [1, 2, 3].

Auch für Software gibt es verwandte Ansätze. **LLMSAN** zerlegt vom Modell beschriebene Datenflusspfade in prüfbare Programmeigenschaften. **GPTAid** leitet Sicherheitsregeln aus dem Code von Programmierschnittstellen ab und prüft sie durch Programmausführung [4, 5]. **VERGE** und **EviGuard** verbinden strukturierte Aussagen mit unterschiedlichen Prüfverfahren [6, 7].

Der geplante Beitrag setzt bei einer konkreten Frage an: **Was verändert sich, wenn zwischen Codeanalyse und strukturierten Claims zunächst ein Bericht entsteht?** Für einen fairen Vergleich erhalten beide Wege dieselben Codeinformationen und müssen dasselbe Ausgabeformat einhalten.

In den bisher betrachteten Arbeiten wurde kein direkt entsprechender Vergleich gefunden, der zugleich die Belege im Code, die Abdeckung wichtiger Aussagen und den Erhalt von Voraussetzungen untersucht. Diese vorläufige Abgrenzung muss vor der Hauptstudie durch eine genauere Literaturprüfung abgesichert werden. Ob P oder D besser abschneidet, bleibt eine offene Forschungsfrage.

3 Forschungsfragen

RQ1: Welche Angaben braucht ein verständlicher und prüfbarer Claim?

Eine Sicherheitsbehauptung ist nur dann eindeutig, wenn erkennbar ist, auf wen oder was sie sich bezieht und unter welchen Bedingungen sie gelten soll. Die erste Forschungsfrage untersucht deshalb, welche Angaben ein Claim enthalten muss und welche Beziehungen zwischen Claims wichtig sind. Daraus soll sich auch ableiten lassen, welche konkrete Prüfung zur Beurteilung einer Aussage erforderlich wäre.

Als Grundlage dienen die Literatur und die manuelle Untersuchung ausgewählter Entwicklungsfälle. Das Ergebnis ist ein *Claimprofil*: eine begründete Festlegung der benötigten Felder. Dazu gehört ein *Codebook*, also ein Leitfaden, der die Felder erklärt und Regeln für schwierige oder mehrdeutige Fälle enthält.

RQ2: Wie unterscheiden sich die beiden Erzeugungswege?

Die zweite Forschungsfrage vergleicht P und D hinsichtlich Qualität und Aufwand. Entscheidend ist, ob ihre Claims durch den verfügbaren Code gestützt werden, wichtige Sicherheitsaspekte abdecken und notwendige Voraussetzungen angemessen ausdrücken. Zusätzlich werden die benötigten Modellaufrufe, Laufzeiten und Kosten erfasst.

Beide Wege werden auf denselben Fällen ausgeführt. Ihre Ausgaben werden anschließend anhand unabhängig erstellter Referenzen bewertet. Berichtet werden die beobachteten Unterschiede und die Unsicherheit dieser Ergebnisse. Ein Gewinner wird nicht vorab festgelegt.

RQ3: Welche Darstellungsfehler erschweren eine spätere Prüfung?

Die dritte Forschungsfrage betrachtet Fehler, die beim Formulieren oder Zerlegen von Aussagen entstehen. Dazu gehören beispielsweise verlorene Bedingungen, zu weit gefasste Aussagen oder ein veränderter Grad an Gewissheit. Untersucht wird, ob solche Fehler dazu führen, dass aus einem Claim eine andere oder unpassende Prüfaufgabe abgeleitet wird.

Auf einem Teil der Fälle werden außerdem Ausgaben mit und ohne ausdrücklich vorgesehene Kontextfelder verglichen. Dieser gezielte Vergleich wird als *Ablation* bezeichnet. Er soll zeigen, ob zusätzliche Felder für Voraussetzungen und ähnliche Angaben helfen, relevante Fehler zu vermeiden.

4 Aufbau des Vergleichs

Fälle und gleiche Ausgangsbedingungen

Ausgangspunkt sind Java-Fälle aus **Vul4J**. Dieser Datensatz enthält bekannte Sicherheitsprobleme, zugehörige Codekorrekturen und Tests, mit denen sich die jeweilige Schwachstelle nachvollziehen lässt [8]. Zunächst werden Fälle aus zwei bis drei passenden Schwachstellenklassen betrachtet. JSPWiki dient weiterhin der Entwicklung; andere Fälle werden für die spätere Evaluation zurückgehalten. Eine reparierte Variante erlaubt die Untersuchung einer bestimmten Korrektur. Sie gilt deshalb noch nicht als insgesamt sicheres Programm.

P und D erhalten exakt dieselben Quellcode- und Konfigurationsdateien. Informationen über die Korrektur, Schwachstellentests, veröffentlichte Sicherheitshinweise und die Einstufung im Datensatz werden ihnen nicht zusätzlich gegeben. Bei P sieht das Modell im zweiten Schritt ausschließlich den Bericht sowie allgemeine Anweisungen zum Claimformat. Es kann in diesem Schritt keine weitere Codeanalyse durchführen.

Für jeden Versuch werden Modellversion, Anweisungen, Eingaben, Wiederholungen und Grenzen der Ausgabelänge festgehalten. P benötigt bis zu zwei Modellaufrufe, D zunächst nur einen. Deshalb wird zusätzlich eine D-Variante untersucht, bei der das Modell seine erste Ausgabe einmal überarbeitet. Damit lassen sich vollständige Erzeugungswege mit ihrem jeweiligen Aufwand vergleichen. Der Einfluss des Ausgabeformats allein wird dadurch noch nicht vollständig isoliert.

Ein gemeinsames Format für die Claims

Beide Wege verwenden dieselbe einfache Struktur. Die Konzepte CAE und SACM helfen dabei, Aussagen, begründende Argumente und Belege voneinander zu unterscheiden [9, 10]. Für den Pilotversuch ergibt sich folgendes Format:

Bereich	Welche Angaben werden festgehalten?
Aussage	Der Claim erhält eine Kennung, eine ausformulierte Behauptung und eine vorläufige Kategorie.
Bedingungen und Umfang	Wer ist betroffen, welche Rechte und Voraussetzungen sind nötig, und wann gilt die Aussage? Auch Verneinungen und Unsicherheit bleiben erhalten.
Herkunft und Beziehungen	Codeverweise und, bei P, Berichtszitate zeigen die Herkunft. Beziehungen zu anderen Claims werden ausdrücklich angegeben.
Prüfaufgabe	Was müsste geprüft werden, und welche Belege fehlen dafür? Diese Aufgabe bleibt von der Behauptung getrennt.

Eine genannte Voraussetzung gilt nicht automatisch als erfüllt. Die Aussage, ergänzende Informationen und die Prüfaufgabe bleiben deshalb getrennt. Eine vollständige Umsetzung von SACM oder eine Übersetzung der Claims in die formale Beweissprache Lean ist nicht vorgesehen.

Wer übernimmt welche Aufgabe?

Die Python-Skripte stellen die Eingaben zusammen, senden die Anfragen an das Modell und speichern dessen Antworten. Das Sprachmodell schreibt den Bericht beziehungsweise die Claims. Die Software prüft danach das Format und bestimmte Verweise. Menschen erstellen die unabhängigen Vergleichsgrundlagen und bewerten die Bedeutung der Aussagen. Der Prototyp entscheidet also nicht automatisch, ob ein Sicherheitsproblem tatsächlich ausnutzbar ist.

Die sechs Kontextfelder in einfachen Worten

Die folgenden Fragen sollen verhindern, dass beim Formulieren wichtige Bedingungen verloren gehen. Im gespeicherten Format haben sie kurze englische Namen. Die Angaben müssen zur vollständigen Aussage passen und dürfen keine neuen Tatsachen erfinden.

Feld	Welche Frage beantwortet es?
actor	Wer handelt oder ist betroffen? Sind bestimmte Rechte oder eine Anmeldung ausdrücklich genannt?
preconditions	Was muss vorausgesetzt werden, damit die Aussage gilt, etwa eine bestimmte Konfiguration oder kontrollierbare Eingabedaten?
negation	Was wird ausdrücklich verneint, etwa „es findet keine Prüfung statt“?
modality	Wird etwas als möglich, sicher oder eingeschränkt behauptet, etwa „könnte“?
quantifier	Geht es um alle, einige oder nur bestimmte Objekte oder Fälle?
scope	Auf welchen Codepfad, welche Version oder welchen sonstigen Bereich ist die Aussage begrenzt?

Eine nicht genannte Information bleibt null. Dagegen werden „keine Anmeldung erforderlich“ und „ob eine Anmeldung nötig ist, ist unbekannt“ als Text festgehalten. Diese drei Situationen bedeuten etwas Unterschiedliches. Wie die Felder in einer echten Ausgabe aussehen, zeigt das folgende Beispiel.

Ein echtes Beispiel: ZIP-Dateien entpacken

Das folgende Beispiel stammt aus dem abgeschlossenen Pilotversuch: **VUL4J-43, Eclipse RDF4J, verwundbare Variante, erste Wiederholung**. Code und Modellausgaben sind vorhanden. Die deutschen Erläuterungen fassen die englischen Ausgaben zusammen; eine unabhängige menschliche Bewertung liegt dazu noch nicht vor.

Beim Entpacken einer ZIP-Datei muss das Programm entscheiden, wo es jeden enthaltenen Eintrag ablegt. Im bereitgestellten Code von `ZipUtil.java` steht in den Zeilen 93–104 der folgende Ausschnitt. Lediglich die Einrückung wurde für die Darstellung angepasst:

```
File outFile = new File(destDir, entry.getName());

if (entry.isDirectory()) {
    outFile.mkdirs();
}
else {
    outFile.getParentFile().mkdirs();

    try (InputStream in = zipFile.getInputStream(entry)) {
        IOUtil.writeStream(in, outFile);
    }
}
```

`destDir` ist das vorgesehene Zielverzeichnis. `entry.getName()` liefert den Namen eines Eintrags aus dem ZIP-Archiv. Aus beiden wird ein Dateipfad gebildet. Danach werden Verzeichnisse angelegt oder die enthaltenen Daten an die Schreibfunktion übergeben.

Warum ist das sicherheitsrelevant? Ein Dateiname kann auch Pfadbestandteile wie `../` enthalten. Als vereinfachtes Anschauungsbeispiel führt der Pfad `ausgabe/ andere.txt` eine Ebene aus `ausgabe` heraus. Die Sicherheitsfrage lautet deshalb: **Wird verhindert, dass Einträge außerhalb des vorgesehenen Verzeichnisses geschrieben werden?** Dieses Pfadbeispiel erklärt die Fragestellung; es ist kein hier ausgeführter Angriffstest.

Weg P, erster Aufruf: Das Modell schreibt einen Bericht. Der echte Bericht beschreibt die Bildung des Zielpfads, behauptet eine fehlende Prüfung der Verzeichnisdgrenze und nennt mögliche Folgen. Eine zentrale Passage lautet im Original:

As a result, if an attacker can control the ZIP contents supplied to `extract(...)` (lines 48-75), extraction may overwrite arbitrary files accessible to the process.

Gemeint ist: Wenn ein Angreifer die übergebenen ZIP-Inhalte kontrollieren kann, könnte das Entpacken Dateien überschreiben, auf die der Prozess zugreifen kann. Der Bericht behauptet damit nicht, dass jeder beliebige Benutzer unter allen Umständen jede Datei überschreiben kann.

Weg P, zweiter Aufruf: Ein Modell zerlegt nur diesen Bericht. Es sieht dabei keinen Code. In diesem Lauf entstanden fünf Claims: die Bildung des Zielpfads, die behauptete fehlende Pfadprüfung, die bedingte Möglichkeit des Überschreibens, die Einordnung als ZIP-Slip-Problem und die Abhängigkeit des Risikos von Umgebung und Dateisystemrechten.

So sehen die erzeugten Claims aus

Der dritte P-Claim enthält die eben erläuterte bedingte Aussage. Der folgende Ausschnitt zeigt echte Feldinhalte; weitere Felder der gespeicherten Ausgabe sind hier zur besseren Lesbarkeit weggelassen:

```
"claim_id": "C03",
"proposition": "If an attacker can control the ZIP contents supplied to 'extract(...)', extraction may
    overwrite arbitrary files accessible to the process.",
"context": {
  "actor": "an attacker",
  "preconditions": "the attacker can control the ZIP contents supplied to 'extract(...)'",
  "negation": null,
  "modality": "may",
  "quantifier": null,
  "scope": "'extract(...)' (lines 48-75)"
}
```

Die `proposition` ist die vollständige Behauptung. Das Objekt `context` schreibt ihre Bedingungen und Einschränkungen zusätzlich einzeln auf: Wer handelt? Was muss vorausgesetzt werden? Ist etwas nur möglich? Worauf bezieht sich die Aussage? `null` bedeutet dabei „nicht angegeben“, nicht „falsch“ oder „geprüft und nicht vorhanden“.

Die getrennten Felder sind ein zusätzlicher Fragenbogen zur Aussage. Sie sollen helfen, wichtige Einschränkungen beim Formulieren zu erhalten. Aus dem Original darf nicht einfach „Ein Angreifer überschreibt beliebige Dateien“ werden. Dann gingen die Voraussetzung, die Möglichkeit statt Gewissheit und die Begrenzung durch Prozessrechte verloren. Auch mit ausgefüllten Feldern muss der Aussagesatz selbst vollständig bleiben. Im echten Beispiel steht die Beschränkung durch Prozessrechte im Satz, aber nicht ausdrücklich im Feld `scope`; die Felder sind also ebenfalls zu beurteilen.

Weg D: Das Modell erzeugt Claims unmittelbar aus demselben Code. In diesem Lauf entstanden sechs Claims. Der erste beschreibt, wie `writeEntry` den Zielpfad bildet und Verzeichnisse beziehungsweise Dateien verarbeitet. Die übrigen Claims beschreiben den Ablauf von `extract` sowie Hilfsfunktionen zum Lesen und Übertragen von Daten. Die sechs Aussagesätze benennen die fehlende Prüfung der Zielverzeichnisgrenze nicht ausdrücklich.

Das zeigt einen möglichen Unterschied, noch keinen Sieger: D hat hier mehr Claims erzeugt, P benennt das mögliche Sicherheitsproblem deutlicher. Ob die Aussagen richtig und für die Sicherheitsfrage relevant sind, muss anhand einer unabhängig erstellten Codereferenz beurteilt werden. Viele zutreffende Beschreibungen nebensächlicher Operationen wären noch keine gute Abdeckung wichtiger Sicherheitsaspekte.

Ein Verweis ist noch kein Beleg. P speichert zu C03 das exakte Berichtszitat und die dort genannten Codepositionen. D nennt beispielsweise den Dateipfad und die Zeilen 90–104 von `writeEntry`. Diese Angaben helfen beim Nachschlagen. Ob die Stelle die gesamte Behauptung trägt, wird erst bei der Bewertung entschieden. Die im Claim enthaltene Prüfaufgabe zu C03 fragt nach dem Überschreiben unter der genannten Voraussetzung und nennt unter anderem benötigte Informationen über Dateisystemrechte. Das ist ein Prüfauftrag, kein bereits durchgeführter Nachweis.

5 Bewertung und Auswertung

Für die Bewertung werden zwei Grundlagen erstellt. Die **Codereferenz** hält fest, welche wichtigen Aussagen der bereitgestellte Code erlaubt und welche Fragen offenbleiben. Sie entsteht, bevor die bewertenden Personen Modellantworten sehen. Die **Reportreferenz** erfasst den Inhalt und die Bedingungen des jeweiligen P-Berichts. So lässt sich unterscheiden, ob ein Claim den Bericht falsch wiedergibt oder ob schon die Berichtsaussage nicht durch den Code gestützt ist.

Wie die Referenzen beim ZIP-Beispiel aussehen könnten

Die folgenden Tabellen sind Erläuterungsentwürfe, keine vorhandenen menschlichen Referenzdaten. Sie wurden für dieses Dokument anhand des bereits besprochenen Falls zusammengestellt. Für die eigentliche Bewertung werden die Referenzen unabhängig nach dem festgelegten Verfahren erstellt; Kenntnis dieser Beispiele wird als Vorwissen dokumentiert.

Eine **Codereferenz** beantwortet: Welche wichtigen Aussagen und offenen Fragen ergeben sich aus dem Code, den auch das Modell sehen durfte? Jeder Eintrag erhält eine eigene Kennung, eine Fundstelle und eine Begründung seiner Relevanz. Ein verkürzter Ausschnitt könnte so aussehen:

ID	Aussage oder offene Frage	Fundstelle und Grenze
K01	Der Zielpfad wird aus dem Zielverzeichnis und dem Namen des ZIP-Eintrags gebildet.	ZipUtil.java, Zeile 93; relevant für den Einfluss des Eintragsnamens.
K02	In writeEntry ist keine Prüfung sichtbar, die den Zielpfad auf das Zielverzeichnis begrenzt.	ZipUtil.java, Zeilen 90–105; Aussage über diese Methode.
K03	Offen: Kann ein Angreifer in der tatsächlichen Anwendung die übergebenen ZIP-Inhalte kontrollieren?	extract, Zeilen 48–75, zeigt die Verarbeitung; der Anwendungskontext fehlt.
K04	Offen: Welche Dateien dürfte der Prozess in seiner Einsatzumgebung überschreiben?	Aufruf der Schreibfunktion in Zeile 102; tatsächliche Prozessrechte fehlen.

Eine **Reportreferenz** beantwortet eine andere Frage: Was behauptet der konkrete Bericht? Sie wird aus dem Bericht erstellt, ohne dessen Aussagen mithilfe des Codes zu verbessern. Dazu gehören exakte Textbelege. Drei beispielhafte Einträge, hier auf Deutsch zusammengefasst, wären:

ID	Im Bericht enthaltene Aussage
R01	Die Methode bildet den Zielpfad aus Zielverzeichnis und Eintragsname und schreibt beziehungsweise erstellt ihn direkt.
R02	Der Bericht behauptet, dass eine Prüfung der Verzeichnisdgrenze und eine entsprechende Prüfung des normalisierten Pfads fehlen.
R03	Wenn ein Angreifer die übergebenen ZIP-Inhalte kontrollieren kann, ist das Überschreiben von Dateien möglich, auf die der Prozess zugreifen kann.

Zu R03 würde beispielsweise das englische Originalzitat auf Seite 4 gehören. Weitere Einträge würden die Einordnung als ZIP Slip und die genannten Einflüsse der Umgebung abdecken. Kennungen wie K01, R01 und C01 sind unabhängig: Die gleiche Nummer bedeutet nicht, dass es sich um die gleiche Aussage handelt.

Danach werden Claims inhaltlich zugeordnet. K01 könnte sowohl durch den ersten P-Claim als auch durch den ersten D-Claim erfasst sein. R03 wird mit dem dritten P-Claim verglichen: Bleiben Voraussetzung, Möglichkeit und Begrenzung durch Prozessrechte erhalten? Mehrere Claims dürfen gemeinsam eine Referenzaussage abdecken; Wiederholungen zählen diese Aussage nicht mehrfach. Teilweise Abdeckung wird mit der konkreten Lücke erfasst.

Wie aus dem Vergleich eine Bewertung wird

Für jeden Claim wird gesondert geprüft, ob der sichtbare Code die vollständige Aussage unter ihren genannten Bedingungen stützt. Die folgenden Urteile veranschaulichen die Regeln; sie sind keine abgeschlossene Annotation der Pilotausgaben:

Urteil	Beispielaussage	Begründung
Belegt	„Der Zielpfad wird mit <code>new File(destDir, entry.getName())</code> gebildet.“	Diese Operation ist in Zeile 93 direkt sichtbar.
Widerlegt	„Die Methode <code>writeEntry</code> prüft vor dem Schreiben, ob der Zielpfad im Zielverzeichnis bleibt.“	Die vollständige sichtbare Methode enthält diese behauptete Prüfung nicht.
Nicht entscheidbar	„Ein nicht angemeldeter Internetnutzer kann über die laufende Anwendung Dateien überschreiben.“	Netzwerkzugang, Anmeldung, Aufrufer und Einsatzumgebung sind im Paket nicht ausreichend sichtbar.

Nicht entscheidbar bedeutet nicht falsch. Für eine Entscheidung fehlen bestimmte Informationen. Diese Lücke wird benannt. Umgekehrt macht ein vorsichtiges Wort wie „könnte“ eine Aussage nicht automatisch richtig: Auch eine behauptete Möglichkeit braucht einen nachvollziehbaren Weg und passende Voraussetzungen.

Bei P wird zusätzlich die **Treue zum Bericht** geprüft. Wenn der Bericht einen Angriff nur bei kontrollierbaren ZIP-Inhalten für möglich hält und der Claim diese Einschränkung erhält, kann die Extraktion bedeutungstreu sein. Das gilt auch dann, wenn die Aussage selbst mit dem Code nicht entscheidbar ist. Lässt der Claim die Voraussetzung weg oder macht er aus der Möglichkeit eine Gewissheit, liegt dagegen eine Bedeutungsänderung vor.

Auch die **Prüfaufgabe** wird bewertet. Beim dritten P-Claim wäre die passende Frage, ob kontrollierbare Eintragsnamen unter den genannten Bedingungen zu einem Schreiben außerhalb des Zielverzeichnisses führen können. Dafür könnten Informationen zum gesamten Schreibpfad und zu den Prozessrechten benötigt werden. Eine Aufgabe, die lediglich prüft, ob sich die ZIP-Datei öffnen lässt, würde diese Behauptung nicht beantworten. Benötigte Belege und zusätzliche Annahmen sind deshalb keine bereits vorhandenen Beweise.

Für die spätere Auswertung entsteht so pro Fall ein Vergleich:

1. Welche relevanten Referenzaussagen erfassen P und D vollständig, teilweise oder gar nicht? Offene Fragen werden von belegbaren Aussagen getrennt betrachtet.
2. Welche Claims sind belegt, widerlegt oder nicht entscheidbar? Welche drücken zu viel oder zu wenig Gewissheit aus?
3. Wo verändert P die Bedeutung des Berichts, und wie gut passen die abgeleiteten Prüfaufgaben zu den Claims?
4. Welchen Aufwand verursachen die vollständigen Wege einschließlich Modellaufrufen und menschlicher Bewertungszeit?

Die getrennten Ergebnisse sollen zeigen, *warum* ein Weg besser oder schlechter funktioniert. Ein technischer Formatfehler wird separat erfasst und nicht als fachlich falscher Claim gezählt. Ein festgelegter Teilbestand wird von zwei Menschen unabhängig beurteilt, möglichst ohne Kenntnis der Route. Die ersten Urteile bleiben erhalten; Unterschiede werden erst danach besprochen. Ein weiteres Sprachmodell ersetzt diese Unabhängigkeit nicht.

P und D werden innerhalb desselben Codefalls verglichen. Claims, Varianten und Wiederholungen sind keine unabhängigen Fälle. Berichtet werden die Größe der Unterschiede und ihre Unsicherheit; ein statistisch uneindeutiges Ergebnis beweist keine Gleichwertigkeit. Auch eine bekannte Schwachstelle oder ein erfolgreicher Schwachstellentest bestätigt nicht automatisch jede Aussage.

6 Was bisher erarbeitet wurde

Gemeinsames Claimprofil und Regeln für die Bewertung

Das **Claimprofil in Version 0.1** ist umgesetzt und wird von beiden Erzeugungswegen verwendet. Ein gemeinsames Schema legt fest, welche Felder eine Antwort enthalten muss. Das Codebook erklärt, wie diese Felder zu verstehen und auszufüllen sind.

Zusätzlich zur eigentlichen Behauptung werden sechs Aspekte ihres Kontexts erfasst: beteiligte Akteure und Rechte, Voraussetzungen, Verneinungen, der Grad der Gewissheit, Mengenangaben wie „alle“ oder „einige“ und der Geltungsbereich. Das Profil unterscheidet dabei zwischen einer nicht genannten Voraussetzung, einer ausdrücklich verneinten Eigenschaft und einer ausdrücklich unbekannten Information. Diese Unterschiede dürfen beim Erzeugen eines Claims nicht verloren gehen.

Codeverweise, Beziehungen zu anderen Claims und die daraus abgeleitete Prüfaufgabe werden ebenfalls festgehalten. Ob vorhandene Belege die Aussage tatsächlich stützen, wird jedoch erst in der gesonderten menschlichen Bewertung entschieden.

Das Profil wurde anhand von JSPWiki und zwei weiteren Entwicklungsfällen ausgearbeitet. Dazu gehören Beispiele für schwierige Formulierungen und uneindeutige Grenzen zwischen Aussagen. Die ältere JSPWiki-Auswertung mit 13 Claims wurde mit KI-Unterstützung und Vorwissen erstellt. Sie hilft bei der Entwicklung, ist aber keine unabhängige Referenz für die spätere Studie.

Für die menschliche Bewertung liegen bereits ein Ablaufplan und Arbeitsblätter vor. Die zugehörigen Codepakete tragen neutrale Kennungen, damit Fallnamen und Variantenlabels nicht zusätzlich auf die erwartete Bewertung hinweisen. Im Code selbst können allerdings weiterhin Projektmerkmale erkennbar sein. Die wichtigsten verwandten Arbeiten wurden ebenfalls eingeordnet; die Prüfung, wie neu der geplante Beitrag gegenüber der gesamten Literatur ist, bleibt offen.

Ein ausführbarer Forschungsprototyp

Ein kleiner Python-Prototyp bereitet die Fälle vor, führt beide Erzeugungswege aus und speichert ihre Ergebnisse. Er unterstützt auch die zusätzlichen Vergleiche sowie die Vorbereitung der menschlichen Bewertung. Festgelegte Quellversionen und eine gemeinsame Aufbereitung sorgen dafür, dass P und D dieselben Codeinformationen erhalten.

P verarbeitet alle Befunde eines Berichts gemeinsam. D kann unabhängig davon ausgeführt werden, ob P einen verwendbaren Bericht erzeugt hat. Automatische Prüfungen kontrollieren, ob die Antworten dem vereinbarten Format entsprechen und ob Verweise innerhalb einer Antwort gültig sind. Bei P wird zusätzlich geprüft, ob die angegebenen Zitate exakt im Bericht vorkommen. Diese Kontrollen bewerten die Form der Antwort, nicht die Wahrheit ihrer Aussagen.

Die ursprünglichen Eingaben, Modellanweisungen und Antworten werden zusammen mit ihrer Herkunft gespeichert. Auch leere, fehlerhafte und unvollständige Ergebnisse bleiben nachvollziehbar. Der Versuch wird nicht automatisch so lange wiederholt, bis eine gültige Antwort entsteht. Eine Kostenkontrolle prüft vor jedem Modellaufruf, ob er innerhalb der festgelegten Grenze liegt.

106 Tests ohne Modellaufrufe prüfen unter anderem, ob beide Wege dieselben Quellen verwenden, ob unzulässige Zusatzinformationen aus den Eingaben ausgeschlossen bleiben und ob Fehler korrekt gespeichert werden. Damit ist die technische Grundlage geprüft. Die inhaltliche Bewertung der erzeugten Claims wird dadurch nicht ersetzt.

Erster Pilotversuch: Ausgaben liegen vor, ihre Bewertung folgt

Der erste Pilotversuch umfasst **fünf neue Fälle aus Vul4J**. Zwei Fälle betreffen die Verarbeitung externer Inhalte in XML-Dokumenten, sogenannte XXE-Probleme, in Apache CXF und OpenRefine. Drei weitere Fälle betreffen die Verarbeitung von Pfaden in Plexus Archiver, Eclipse RDF4J und Retrofit.

Für jeden Fall wurde eine verwundbare und eine reparierte Variante vorbereitet. Daraus ergeben sich zehn Codepakete. Beide Erzeugungswege wurden auf jedem Paket zweimal ausgeführt. Die Reihenfolge wechselte so, dass bei jedem Paket einmal P und einmal D zuerst an der Reihe war. Sämtliche Projekte aus der Entwicklung und dem Pilotversuch bleiben von der späteren Evaluation an zurückgehaltenen Fällen ausgeschlossen.

Alle Modellaufrufe verwendeten die festgelegte Version `gpt-5.4-mini-2026-03-17` mit der Reasoning-Einstellung `none`. Für jedes Codepaket wurde zusätzlich die erste D-Ausgabe einmal überarbeitet. Auf vier Codepaketen wurden beide Wege außerdem ohne die gesonderten Kontextfelder ausgeführt. Eine einfache Satzaufteilung und ein an VeriScore angelehntes Verfahren liefern weitere Vergleichsergebnisse für P.

Das VeriScore-basierte Verfahren verwendet einen angepassten, lizenzierten Originalprompt; das vollständige VeriScore-System wird damit nicht nachgebaut. Auch die zusätzliche Überarbeitung von D hat eine begrenzte Aussagekraft: Sie ermöglicht einen Vergleich mit einem zweiten Modellaufruf, gleicht aber nicht automatisch die Anzahl der verarbeiteten oder erzeugten Tokens an. Tokens sind die Texteinheiten, anhand derer unter anderem die API-Nutzung abgerechnet wird.

Versuchsteil	Aufgaben	Formatprüfung bestanden
P: Bericht erzeugen	20	20
P: Claims aus dem Bericht extrahieren	20	16
D: Claims direkt erzeugen	20	19
D: erste Ausgabe einmal überarbeiten	10	10
P ohne gesonderte Kontextfelder	4	3
D ohne gesonderte Kontextfelder	4	4
Satz-/VeriScore-Vergleiche je Bericht	4	4
Gesamt	82	76

Insgesamt wurden 82 Aufgaben mit 122 Modellaufrufen bearbeitet. Die Zahl der Aufrufe ist höher, weil das VeriScore-basierte Verfahren für einzelne Sätze eigene Anfragen stellt. Sechs Ausgaben bestanden die automatische Prüfung nicht. Dabei wurden ungenaue Zitate, unzulässige Unterkategorien, ungültige Zeilenangaben oder ein nicht vorgesehenes Feld festgestellt. Die ursprünglichen Antworten und Fehlermeldungen sind erhalten.

Aus den gemeldeten Tokens ergeben sich für den gesamten Pilotversuch geschätzte API-Kosten von rund **1,09 USD**. Darin sind die zusätzlichen Vergleiche und die fehlerhaften Ausgaben enthalten. Der menschliche Arbeitsaufwand ist damit noch nicht erfasst.

Der Pilot zeigt, dass der technische Ablauf funktioniert und welche konkreten Ausgabeprobleme auftreten. Ob die Claims fachlich richtig, vollständig und ihrem Bericht treu sind, ist noch nicht bewertet. Die Zahl formal gültiger Antworten reicht daher nicht aus, um einen Erzeugungsweg als besser einzustufen. Zudem könnten dem Modell die öffentlichen Fälle bereits bekannt sein. Die gezielt ausgewählten Codepakete bilden nur einen Teil des jeweiligen Projekts ab. Neue Java- oder Schwachstellentests wurden im Rahmen dieses Piloten nicht ausgeführt.

Welche Rolle VeriScore bereits spielt

Für vier P-Berichte wurde zusätzlich ausprobiert, wie andere Verfahren denselben Text zerlegen. Solche Vergleichsverfahren werden als *Baselines* bezeichnet. Hier gibt es drei Möglichkeiten:

Verfahren	Was entsteht aus demselben Bericht?
Unsere P-Extraktion	Claims im gemeinsamen Format, einschließlich Kontextfeldern, Zitaten und Prüfaufgaben.
Einfache Satzaufteilung	Einzelne unveränderte Sätze aus Titel und Bericht. Dafür ist kein weiterer Modellaufruf nötig.
VeriScore-basierte Extraktion	Einzelne vom Modell herausgearbeitete Aussagen. Unser zusätzliches Claimformat wird dabei nicht verlangt.

Damit wird gefragt, ob unser speziell für Sicherheitsbehauptungen entwickeltes Format gegenüber einfacheren oder bereits vorhandenen Verfahren einen erkennbaren Nutzen hat. **VeriScore berechnet in diesem Projekt keinen Wahrheitswert.** Übernommen wird der Extraktionsprompt mit seinen Beispielen, nicht das vollständige System mit Suche und Faktenprüfung. Das Modell erhält jeweils einen markierten Satz und umgebenden Text zum Verständnis. Es soll daraus eigenständig verständliche Aussagen bilden.

Ein echtes Beispiel stammt aus **VUL4J-41, Plexus Archiver, verwundbare Variante, erste Wiederholung**. Dies ist ein anderer Fall als das RDF4J-Beispiel. Der Bericht beschreibt, wie `extractFile(...)` einen Pfad ermittelt, dorthin schreibt und dabei laut Bericht keine ausdrückliche Prüfung der Verzeichnissgrenze vornimmt. Daraus entstanden unter anderem diese VeriScore-basierten Claims, hier auf Deutsch wiedergegeben:

ID	Tatsächlich extrahierte Aussage, übersetzt
V0003	<code>extractFile(...)</code> bestimmt den Pfad jedes Archiveintrags mit <code>FileUtils.resolveFile(dir, entryName)</code> .
V0004	<code>extractFile(...)</code> schreibt anschließend an den ermittelten Pfad.
V0005	<code>extractFile(...)</code> enthält keine ausdrückliche Prüfung, ob der ermittelte Pfad innerhalb von <code>dir</code> bleibt.

Auch diese Ausgaben sind zunächst Behauptungen. In der späteren Bewertung werden ihre Inhalte, ihre Abdeckung des Berichts und ihre Bedeutungstreue verglichen. Dass sie keine Kontextfelder oder Prüfaufgaben enthalten, wird ihnen nicht als Fehler angerechnet; diese Felder wurden hier gar nicht verlangt.

Der Vergleich wurde für VUL4J-15 und VUL4J-41 durchgeführt, jeweils für beide Codevarianten und die erste Wiederholung. Die vier Berichte führten zu insgesamt 44 VeriScore-basierten Modellaufrufen, weil einzelne Sätze getrennt bearbeitet werden. Verwendet wurden ein einfacher eigener Satzteiler, die getrennte Behandlung von Titel und Bericht und dasselbe Pilotmodell. Es handelt sich daher um eine dokumentierte Anpassung, keine exakte Reproduktion der veröffentlichten Methode.

Eine Grenze ist für die Forschungsfrage besonders interessant: Der übernommene Prompt fordert dazu auf, hypothetische Inhalte auszuschließen. Sicherheitsberichte enthalten jedoch oft wichtige Aussagen wie „Wenn ein Angreifer X kontrolliert, könnte Y passieren“. Ob dabei Bedingungen oder ganze Aussagen verloren gehen, muss die Bewertung zeigen. Es wird nicht schon aus der Promptformulierung auf einen tatsächlichen Qualitätsverlust geschlossen.

Wie der Nutzen der Kontextfelder untersucht wird

Die sechs Kontextfelder sind Bestandteil unseres eigenen Formats. Sie sind kein zusätzlicher VeriScore-Schritt. Im ZIP-Beispiel wurden bereits Akteur, Voraussetzung, Verneinung, Möglichkeit, Mengenangabe und Geltungsbereich gezeigt. Nun stellt sich die Frage: **Hilft es dem Modell tatsächlich, diese Angaben zusätzlich einzeln aufzuschreiben?**

Um das zu untersuchen, wurde auf einem Teil des Piloten jeweils eine weitere Ausgabe ohne das Objekt context erzeugt. Dies ist die geplante Kontextfeld-Ablation. Der Unterschied ist bewusst begrenzt:

Mit Kontextfeldern	Ohne Kontextfelder
Die Proposition muss alle wesentlichen Bedingungen enthalten.	Die Proposition muss weiterhin alle wesentlichen Bedingungen enthalten.
Die sechs Aspekte werden zusätzlich in eigenen Feldern festgehalten.	Das Objekt context fehlt vollständig.
Codeverweise, P-Zitate, Beziehungen zu anderen Claims und Prüfaufgaben bleiben erhalten.	Dieselben übrigen Bestandteile bleiben erhalten.

Es werden keine Informationen aus dem Code oder Bericht entfernt. Bei P wird derselbe ursprüngliche Bericht erneut extrahiert. Bei D werden aus denselben Codebytes erneut Claims erzeugt. Nur die Aufforderung, die sechs Aspekte in gesonderten Feldern auszugeben, entfällt aus Format und Anweisungen. Bereits erzeugte Felder werden also nicht einfach nachträglich gelöscht; das Modell erzeugt eine neue Antwort unter der geänderten Vorgabe.

Als Anschauungsbeispiel sollte in beiden Varianten der Satz erhalten bleiben: „Wenn ein Angreifer die ZIP-Inhalte kontrollieren kann, könnte das Entpacken für den Prozess zugängliche Dateien überschreiben.“ Wenn eine Variante stattdessen nur „Ein Angreifer überschreibt Dateien“ liefert, wäre die relevante Bedeutung verändert. Das ist ein Beispiel für das Bewertungskriterium, kein beobachtetes Ergebnis dieses Zusatzversuchs.

Verglichen wird später, ob Voraussetzungen, Rechte, Verneinungen und Unsicherheit mit Zusatzfeldern häufiger richtig erhalten bleiben. Auch unbelegte Ergänzungen und Widersprüche zwischen Satz und Feldern werden betrachtet. Eine vollständig ausgefüllte Struktur allein zählt nicht als bessere Qualität. Ebenso kann eine Ausgabe ohne Zusatzfelder alle wichtigen Einschränkungen korrekt im Satz ausdrücken.

Der Zusatzversuch wurde für dieselben vier Codepakete von VUL4J-15 und VUL4J-41 ausgeführt. Drei von vier P-Ausgaben und alle vier D-Ausgaben ohne Kontextfelder bestanden die Formatprüfung. Für den zuvor gezeigten RDF4J-Fall VUL4J-43 gibt es diesen Zusatzversuch nicht. Dort dienen die vorhandenen Kontextfelder nur als konkretes Beispiel für das Format.

Es liegen somit zwei unterschiedliche Vergleiche vor: Der **VeriScore-Vergleich** untersucht verschiedene Verfahren zur Zerlegung desselben Berichts. Der **Kontextfeld-Vergleich** untersucht innerhalb unseres Formats, ob das zusätzliche Aufschreiben der Bedingungen hilft. Beide Vergleiche wurden technisch durchgeführt. Ob sie einen Qualitätsunterschied zeigen, bleibt bis zur menschlichen Bewertung offen.

7 Nächste Schritte und Abstimmung

Zuerst die Grundlagen für eine unabhängige Bewertung schaffen

Im nächsten Schritt werden die zehn Codepakete von Menschen untersucht. Dabei halten sie fest, welche wichtigen Aussagen der sichtbare Code erlaubt und welche Fragen er offenlässt. Diese Codereferenzen müssen entstehen, bevor die Personen Modellantworten lesen. Danach werden die tatsächlich erzeugten P-Berichte getrennt von den daraus extrahierten Claims ausgewertet. So wird vermieden, dass die Modellantworten schon vorab bestimmen, was als richtige oder wichtige Aussage gilt.

Bei dieser Arbeit werden Vorwissen, verwendete Hilfsmittel und die tatsächlich benötigte Arbeitszeit dokumentiert. Anschließend werden die Claims anhand des gemeinsamen Leitfadens bewertet. Für vier Codepakete, nämlich beide Varianten von CXF und Plexus Archiver, ist eine zweite unabhängige menschliche Bewertung vorgesehen. Die Personen geben ihre Urteile zunächst getrennt ab. Erst danach besprechen sie Unterschiede und halten eine gemeinsame Entscheidung fest.

Den Pilot auswerten und daraus die Hauptstudie ableiten

Die Auswertung soll zeigen, welche Aussagen vom Code gestützt werden, welche wichtigen Inhalte fehlen und an welchen Stellen Bedingungen oder Bedeutungen verändert wurden. Dabei wird auch geprüft, ob sich aus den Claims sinnvolle Prüfaufgaben ableiten lassen. Technisch fehlgeschlagene Ausgaben werden getrennt von Aussagen erfasst, deren Wahrheit sich anhand des verfügbaren Codes nicht entscheiden lässt.

Zusätzlich werden der gesamte Aufwand beider Erzeugungswege und die menschliche Bewertungszeit betrachtet. Die einmalige Überarbeitung von D und der Vergleich mit und ohne Kontextfelder werden als eigene Versuche ausgewertet. Ergebnisse mehrerer Varianten und Wiederholungen werden zunächst innerhalb desselben Falls zusammengeführt.

Erst auf dieser Grundlage werden Umfang und Zeitplan der Hauptstudie festgelegt. Dazu gehören die Auswahl bisher zurückgehaltener Fälle, eine begründete Fallzahl und ein genauer statistischer Auswertungsplan. Auch die Literaturabgrenzung wird vervollständigt. Vor Beginn der Hauptstudie werden Schema, Modellanweisungen und Bewertungsregeln verbindlich festgehalten. Änderungen, die sich aus den Pilotergebnissen ergeben, müssen ausdrücklich begründet werden.

Ausgewählte Literatur

- [1] M. Wanner, B. Van Durme und M. Dredze (2025). *DnDScore: Decontextualization and Decomposition for Factuality Verification in Long-Form Text Generation*. EMNLP, S. 23609–23626. doi:10.18653/v1/2025.emnlp-main.1205.
- [2] M. Wanner, S. Ebner, Z. Jiang, M. Dredze und B. Van Durme (2024). *A Closer Look at Claim Decomposition*. Verwendete Fassung: arXiv:2403.11903. <https://arxiv.org/abs/2403.11903>.
- [3] Y. Song, Y. Kim und M. Iyyer (2024). *VeriScore: Evaluating the Factuality of Verifiable Claims in Long-Form Text Generation*. Findings of EMNLP, S. 9447–9474. doi:10.18653/v1/2024.findings-emnlp.552.
- [4] C. Wang, W. Zhang, Z. Su, X. Xu und X. Zhang (2024). *Sanitizing Large Language Models in Bug Detection with Data-Flow*. Findings of EMNLP, S. 3790–3805. doi:10.18653/v1/2024.findings-emnlp.217.
- [5] J. Liu, Y. Yang, K. Chen und M. Lin (2025). *Generating API Parameter Security Rules with LLM for API Misuse Detection*. NDSS. doi:10.14722/ndss.2025.230465.
- [6] V. Singh, D. Cassel, N. Weir, N. Feng und S. Bayless (2026). *VERGE: Formal Refinement and Guidance Engine for Verifiable LLM Reasoning*. Verwendete Fassung: arXiv:2601.20055v2. <https://arxiv.org/abs/2601.20055v2>.
- [7] H. Zhou, H. Lei und M. Yang (2026). *EviGuard: Machine-Verifiable Evidence Grounding for LLM-Based Industrial Incident Reasoning*. Applied Sciences, 16(18), 8925. doi:10.3390/app16188925.
- [8] Q.-C. Bui, R. Scandariato und N. E. Díaz Ferreyra (2022). *Vul4J: A Dataset of Reproducible Java Vulnerabilities Geared Towards the Study of Program Repair Techniques*. MSR, S. 464–468. doi:10.1145/3524842.3528482.
- [9] Adelard (o. J.). *Claims, Arguments and Evidence (CAE)*. Konzeptionelle Dokumentation; abgerufen am 03.10.2026. <https://www.adelard.com/asce/cae/>.
- [10] Object Management Group (2021). *Structured Assurance Case Metamodel (SACM), Version 2.2*. Normative Spezifikation und Metamodell. <https://www.omg.org/spec/SACM/2.2>.